A cutoff-independent enstrophy bound for the $\sigma = 3$ dispersive dyadic shell model: kernel-checked conservation of the second invariant along solutions, CVODE controls, and an illustration on The Well data

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-25

AI-assistance disclosure. This preprint, its Lean code, its Rust and Python code and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements for adequacy, and no human referee has checked the hand proof. The Lean files were checked by the agent that wrote them and recompiled by a second agent instance. A separate AI-agent instance then ran an independent audit (`AUDIT.md`): it recompiled the Lean files, planted its own `sorry` and axiom (both caught by the axiom audit), reran the numerics and the unit tests, and re-derived the hand proof. That audit is not a human audit.

Abstract

We study the truncated, complexified dyadic shell model of the QuantumFluids project, with shells $n = 0, \dots, N+1$, wavenumbers $k_n = 2^n$ and a linear dispersive term $-i\omega_n v_n$. Its second invariant $H = \sum 2^{-n}\omega_n |v_n|^2 + \sum 2^{-n} k_n \operatorname{Im}(\overline{v_n}^2 v_{n+1})$ is conserved for *every* real dispersion law ω_n. For $\omega_n = D k_n^3$ ("$\sigma = 3$"), the quadratic part of H equals $2D\Omega$, twice D times the enstrophy, and the cubic part is bounded by $S^{3/2}$ with $S = \sum |v_n|^2$.

Tier A (kernel-checked in Lean v4.34.0-rc2 with the project's Mathlib build; axioms within {`propext`, `Classical.choice`, `Quot.sound`}; planted `sorry` and planted-axiom controls caught). We prove the rate identity of H along the full vector field, the chain rule, and constancy of H and S on $[0, T]$ along any solution. As a consequence, $2D\Omega(t) \le H_3(0) + (2E(0))^{3/2}$ for every $t \in [0, T]$ and every cutoff N. Existence of solutions is a hypothesis, not a theorem, and N is fixed throughout.

Tier C. A step-by-step human proof for undergraduates. We also derive the corollary $\Omega(t) \le \Omega(0) + S(0)^{3/2}/D$, which is cutoff-uniform only for initial data with N-uniform $\Omega(0)$ and $S(0)$. On real data with $D = 0$ the model reduces to Waleffe's dyadic Burgers chain [L].

Tier X. Floating-point only. A new rusty-SUNDIALS module integrates the model with CVODE (BDF). Conservation converges with tolerance, while a perturbed coupling breaks it at $O(10^{-3})$. Every $\sigma = 3$ run ($n_{\max} \le 7$) stays within the bound, i.e. the ratio $2D\Omega(t)/(H_3(0) + S(0)^{3/2})$ is at most 1. Its maximum is 0.360 on decaying data, 0.631 on data-initialised runs, and 0.995 on near-extremal top-shell data, where the supremum equals the $t = 0$ value to within 3.2×10^{-7}. A $\sigma = 2$ control grows like $2^{n_{\max}}$. Spectra from the HuggingFace dataset `polymathic-ai/turbulent_radiative_layer_2D` (The Well, CC-BY-4.0) initialise 24 runs, all within the bound.

This result is a short corollary of the owner's `dispersive_norm_le`. What is new is the Lean proof of the dispersive cross-term cancellation and of the chain rule along solutions, which the owner's files had listed as not proved.

Tiers. [A]: kernel-checked Lean, with axioms within the allowlist and planted controls caught. [L]: a cited source, quoted in the ledger (prose as extracted, mathematics transcribed linearly). [C]: an argument or human proof that is not machine-checked. [X]: floating-point exploration, which never supports a claim. Identifiers such as `SE-A-0005` refer to `ledger/ledger.json` (Section 9). A Tier A tag attaches to

the Lean statement, never to its physical gloss. Unlike our triangular-lattice preprints [7, 8], nothing here depends on the openai/math `universal_energy_minimum`. The conditions that do apply are stated in Section 7.

1 The model

The QuantumFluids repository [1] defines a complexified Katz–Pavlović-type nonlinearity `shellBc` [2]. For a sequence $v : \mathbb{N} \to \mathbb{C}$,

$$B_0(v) = -k_0\,\overline{v_0}\,v_1, \qquad B_n(v) = k_{n-1}v_{n-1}^2 - k_n\,\overline{v_n}\,v_{n+1} \quad (n \geq 1). \tag{1}$$

We add a dispersive rotation and truncate at shell $N+1$:

$$\dot{v}_n = F_n(v) := B_n(v) - i\,\omega_n v_n \quad (0 \leq n \leq N+1), \qquad v_{N+2} \equiv 0. \tag{2}$$

Throughout, $k_n = 2^n$, and "σ" denotes the law $\omega_n = Dk_n^\sigma$ with a real constant D. The quantities are

$$S = \sum_{n\leq N+1} |v_n|^2 = 2E, \qquad \Omega = \tfrac{1}{2}\sum_{n\leq N+1} k_n^2|v_n|^2, \qquad H_\sigma = \sum_{n\leq N+1} 2^{-n}\omega_n|v_n|^2 + \sum_{n\leq N} 2^{-n}k_n\,T_n,$$

with $T_n = \mathrm{Im}(\overline{v_n}^2 v_{n+1})$ and $R_n = \mathrm{Re}(\overline{v_n}^2 v_{n+1})$. The owner's design memo proposes $\sigma = 3$ as the regulator that would control enstrophy. It notes that quantum pressure, $\sigma = 2$, controls only $\sum k_n|v_n|^2$ [1]. The model is dimensionless. Under $v \mapsto \lambda v$, $t \mapsto t/\lambda$ (that is, $w(s) = \lambda v(\lambda s)$, so $\dot{w} = B(w) - i(\lambda D)k^3 w$) the nonlinearity is invariant, $D \mapsto \lambda D$ and $S \mapsto \lambda^2 S$, so only $D/\sqrt{S}$ is meaningful [C].

2 Human mathematical demonstration

This section is [C]: a proof on paper, written to be checked line by line. Each step is followed by the Lean statement that the kernel checked [A]. The full statements are listed in Section 3. We use two facts about complex numbers: $\frac{d}{dt}|z|^2 = 2\,\mathrm{Re}(\overline{z}\,\dot{z})$, and $\mathrm{Re}(\overline{w}) = \mathrm{Re}(w)$.

Step 1: the rotation does no work. $\mathrm{Re}\big(\overline{v_n}\cdot(-i\omega_n v_n)\big) = \mathrm{Re}(-i\omega_n|v_n|^2) = 0$, because $-i\omega_n|v_n|^2$ is purely imaginary. Hence $\mathrm{Re}(\overline{v_n}F_n) = \mathrm{Re}(\overline{v_n}B_n)$. *Lean:* `re_conj_mul_F`.

```
-- Autoresearch_EnstrophyFlow.lean lines 117-118
theorem re_conj_mul_F (k ω : ℕ → ℝ) (v : ℕ → ℂ) (n : ℕ) :
    ((starRingEnd ℂ) (v n) * F k ω v n).re = ((starRingEnd ℂ) (v n) * shellBc k n v).re := ...
```

Step 2: energy is conserved. For $n \geq 1$, $\mathrm{Re}(\overline{v_n}B_n) = k_{n-1}\mathrm{Re}(\overline{v_n}v_{n-1}^2) - k_n\mathrm{Re}(\overline{v_n}^2 v_{n+1})$. The first real part is Re of the conjugate of $\overline{v_{n-1}}^2 v_n$, so it equals R_{n-1}. Thus $\mathrm{Re}(\overline{v_n}B_n) = k_{n-1}R_{n-1} - k_nR_n$ for $n \geq 1$, and $\mathrm{Re}(\overline{v_0}B_0) = -k_0R_0$. The sum over $n = 0,\ldots,N+1$ telescopes to $-k_{N+1}R_{N+1}$, and R_{N+1} contains the factor $v_{N+2} = 0$. By Step 1, $\dot{S} = 2\sum \mathrm{Re}(\overline{v_n}F_n) = 0$. *Lean:* `re_B_zero`, `re_B_succ`, `energy_rate_zero`. The last of these reuses the owner's `shellBc_energy_conservation`.

```
-- Autoresearch_EnstrophyFlow.lean lines 137-138
theorem re_B_succ (k : ℕ → ℝ) (v : ℕ → ℂ) (m : ℕ) :
    ((starRingEnd ℂ) (v (m + 1)) * shellBc k (m + 1) v).re = k m * R v m - k (m + 1) * R v (m + 1) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 200-201
theorem energy_rate_zero (k ω : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hbc : v (N + 2) = 0) :
    ∑ n ∈ Finset.range (N + 2), 2 * ((starRingEnd ℂ) (v n) * F k ω v n).re = 0 := ...
```

Step 3: rate of the cubic part. By the product rule, $\dot{T}_n = \mathrm{Im}\big(2\overline{v_n}\,\overline{\dot{v}_n}\,v_{n+1} + \overline{v_n}^2\dot{v}_{n+1}\big)$. Insert $\dot{v} = B - i\omega v$. The dispersive pieces give $\mathrm{Im}(2\overline{v_n}\cdot i\omega_n\overline{v_n}\cdot v_{n+1}) = 2\omega_n R_n$, since $\overline{-i\omega_n v_n} = i\omega_n\overline{v_n}$, and $\mathrm{Im}(\overline{v_n}^2\cdot(-i\omega_{n+1}v_{n+1})) = -\omega_{n+1}R_n$. Hence

$$\dot{T}_n = dT_n + (2\omega_n - \omega_{n+1})R_n, \tag{3}$$

where dT_n is the same expression with B in place of F. The owner proved $dT_0 = -k_1X_0$ and $dT_{m+1} = 2k_mX_m - k_{m+2}X_{m+1}$, with $X_m = \mathrm{Im}(\overline{v_m}^2\overline{v_{m+1}}v_{m+2})$. For weights g_n with $g_{n+1}\cdot 2k_n = g_nk_{n+1}$, the coefficient of X_m in $\sum_{n\le N} g_n\,dT_n$ is $2g_{m+1}k_m - g_mk_{m+1} = 0$. The only survivor is $-g_Nk_{N+1}X_N$, and X_N contains $v_{N+2} = 0$. For $g_n = 2^{-n}k_n$ the weight condition reads $2^{-n-1}k_{n+1}\cdot 2k_n = 2^{-n}k_nk_{n+1}$, which is true. *Lean:* `dT_F` (new) and the owner's `sum_dT` and `weights_ok`.

```
-- Autoresearch_EnstrophyFlow.lean lines 124-127
theorem dT_F (k ω : ℕ → ℝ) (v : ℕ → ℂ) (n : ℕ) :
    (2 * (starRingEnd ℂ) (v n) * (starRingEnd ℂ) (F k ω v n) * v (n + 1)
        + (starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * F k ω v (n + 1)).im
      = dT k v n + (2 * ω n - ω (n + 1)) * R v n := ...
```

```
-- owner ShellHamiltonian.lean lines 72-73
theorem weights_ok (k : ℕ → ℝ) (n : ℕ) :
    ((1 / 2 : ℝ) ^ (n + 1) * k (n + 1)) * (2 * k n) = ((1 / 2 : ℝ) ^ n * k n) * k (n + 1) := ...
```

Step 4: the quadratic part absorbs the cross terms. Let q_n be the quadratic weights. By Steps 1–2, $\frac{d}{dt}\sum q_n|v_n|^2 = 2\sum_{n\le N+1} q_n(k_{n-1}R_{n-1} - k_nR_n)$ (no R_{-1} term for $n = 0$). Add the cross terms $\sum_{n\le N} g_n(2\omega_n - \omega_{n+1})R_n$ from (3). The coefficient of R_m, for $m \le N$, is

$$2q_{m+1}k_m - 2q_mk_m + g_m(2\omega_m - \omega_{m+1}).$$

It vanishes exactly when $2q_{m+1}k_m + g_m(2\omega_m - \omega_{m+1}) = 2q_mk_m$. For $q_n = 2^{-n}\omega_n$, $g_n = 2^{-n}k_n$ the left side is $2^{-m}\omega_{m+1}k_m + 2^{1-m}k_m\omega_m - 2^{-m}k_m\omega_{m+1} = 2\cdot 2^{-m}\omega_mk_m$, which is the right side *for every* ω. The coefficient of R_{N+1} is $-2q_{N+1}k_{N+1}$, and $R_{N+1} = 0$. Together with Step 3, $\dot{H} = 0$ for every real dispersion law. So the exponent σ plays no role in conservation. *Lean:* `quad_tel`, `cross_weights_ok`, `rate_zero`, `ham_rate_zero`.

```
-- Autoresearch_EnstrophyFlow.lean lines 143-147
theorem quad_tel (k ω q g : ℕ → ℝ) (v : ℕ → ℂ)
    (hw : ∀ n, 2 * q (n + 1) * k n + g n * (2 * ω n - ω (n + 1)) = 2 * q n * k n) :
    ∀ M : ℕ, ∑ n ∈ Finset.range (M + 1), q n * (2 * ((starRingEnd ℂ) (v n) * shellBc k n v).re)
        + ∑ n ∈ Finset.range M, g n * ((2 * ω n - ω (n + 1)) * R v n)
      = -(2 * q M * k M * R v M) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 188-190
theorem cross_weights_ok (k ω : ℕ → ℝ) (n : ℕ) :
    2 * ((1 / 2 : ℝ) ^ (n + 1) * ω (n + 1)) * k n + ((1 / 2 : ℝ) ^ n * k n) * (2 * ω n - ω (n + 1))
      = 2 * ((1 / 2 : ℝ) ^ n * ω n) * k n := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 159-163
theorem rate_zero (k ω q g : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ)
    (hg : ∀ n, g (n + 1) * (2 * k n) = g n * k (n + 1))
    (hw : ∀ n, 2 * q (n + 1) * k n + g n * (2 * ω n - ω (n + 1)) = 2 * q n * k n)
    (hbc : v (N + 2) = 0) :
    dHam q g N v (F k ω v) = 0 := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 195-196
theorem ham_rate_zero (k ω : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hbc : v (N + 2) = 0) :
    dHam (fun n => (1 / 2 : ℝ) ^ n * ω n) (fun n => (1 / 2 : ℝ) ^ n * k n) N v (F k ω v) = 0 := ...
```

Step 5: from rates to constancy. If $u(t)$ solves (2) on $[0,T]$, the chain rule gives $\frac{d}{dt}H(u(t)) = dH_{u(t)}[F(u(t))]$, and this is 0 by Step 4. A real function on $[0,T]$ that is continuous and has zero derivative everywhere is constant (mean value theorem). Hence $H(u(t)) =$

$H(u(0))$, and likewise $S(u(t)) = S(u(0))$. *Lean:* `hasDerivAt_hamG`, `const_on_Icc` (from Mathlib's `constant_of_has_deriv_right_zero`), `ham_conserved`, `energy_conserved`.

```
-- Autoresearch_EnstrophyFlow.lean lines 255-257
theorem hasDerivAt_hamG (q g : ℕ → ℝ) (N : ℕ) (u : ℝ → ℕ → ℂ) (u' : ℕ → ℂ) (t : ℝ)
    (hu : ∀ n, n < N + 2 → HasDerivAt (fun s => u s n) (u' n) t) :
    HasDerivAt (fun s => hamG q g N (u s)) (dHam q g N (u t) u') t := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 272-275
theorem const_on_Icc (Φ : (ℕ → ℂ) → ℝ) (Φ' : (ℕ → ℂ) → ℝ) (u : ℝ → ℕ → ℂ) (T : ℝ)
    (hd : ∀ t ∈ Set.Icc (0 : ℝ) T, HasDerivAt (fun s => Φ (u s)) (Φ' (u t)) t)
    (hz : ∀ t ∈ Set.Icc (0 : ℝ) T, Φ' (u t) = 0) :
    ∀ t ∈ Set.Icc (0 : ℝ) T, Φ (u t) = Φ (u 0) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 287-293
theorem ham_conserved (k ω : ℕ → ℝ) (N : ℕ) (u : ℝ → ℕ → ℂ) (T : ℝ)
    (hsol : ∀ t ∈ Set.Icc (0 : ℝ) T, ∀ n, n < N + 2 →
      HasDerivAt (fun s => u s n) (F k ω (u t) n) t)
    (hbc : ∀ t ∈ Set.Icc (0 : ℝ) T, u t (N + 2) = 0) :
    ∀ t ∈ Set.Icc (0 : ℝ) T,
      hamG (fun n => (1 / 2 : ℝ) ^ n * ω n) (fun n => (1 / 2 : ℝ) ^ n * k n) N (u t)
        = hamG (fun n => (1 / 2 : ℝ) ^ n * ω n) (fun n => (1 / 2 : ℝ) ^ n * k n) N (u 0) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 300-304
theorem energy_conserved (k ω : ℕ → ℝ) (N : ℕ) (u : ℝ → ℕ → ℂ) (T : ℝ)
    (hsol : ∀ t ∈ Set.Icc (0 : ℝ) T, ∀ n, n < N + 2 →
      HasDerivAt (fun s => u s n) (F k ω (u t) n) t)
    (hbc : ∀ t ∈ Set.Icc (0 : ℝ) T, u t (N + 2) = 0) :
    ∀ t ∈ Set.Icc (0 : ℝ) T, energy N (u t) = energy N (u 0) := ...
```

Step 6: the inequality, and why $\sigma = 3$. With $\omega_n = Dk_n^3$, $q_n = 2^{-n}D\,8^n = D\,4^n = Dk_n^2$, so the quadratic part of H_3 is $\sum Dk_n^2|v_n|^2 = 2D\Omega$. For the cubic part, $|T_n| \le |v_n|^2|v_{n+1}| \le |v_n|^2\sqrt{S}$, and $g_n = 1$, so $|\sum_{n\le N} g_nT_n| \le \sqrt{S}\sum_{n\le N}|v_n|^2 \le S^{3/2}$. Therefore

$$2D\Omega = H_3 - \sum g_nT_n \le H_3 + S^{3/2} = H_3 + (2E)^{3/2}.$$

By Step 5, at any time $t \in [0,T]$,

$$2D\,\Omega(t) \;\le\; H_3(t) + (2E(t))^{3/2} \;=\; H_3(0) + (2E(0))^{3/2}. \tag{4}$$

Lean: the owner's `cubic_bound` and `dispersive_norm_le`; `enstrophy_bound` (pointwise), `enstrophy_bound_along_flow`, `enstrophy_le_along_flow` (divided by $2D$ for $D > 0$).

```
-- owner ShellHamiltonian.lean lines 84-87
theorem cubic_bound (c : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hc : ∀ n, |c n| ≤ 1) :
    |∑ n ∈ Finset.range (N + 1), c n * ((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).im|
      ≤ Real.sqrt (∑ n ∈ Finset.range (N + 2), Complex.normSq (v n))
        * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n) := ...
```

```
-- owner ShellHamiltonian.lean lines 126-131
theorem dispersive_norm_le (q c : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (Hval : ℝ) (hc : ∀ n, |c n| ≤ 1)
    (hH : Hval = ∑ n ∈ Finset.range (N + 2), q n * Complex.normSq (v n)
      + ∑ n ∈ Finset.range (N + 1), c n * ((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).im) :
    ∑ n ∈ Finset.range (N + 2), q n * Complex.normSq (v n)
      ≤ Hval + Real.sqrt (∑ n ∈ Finset.range (N + 2), Complex.normSq (v n))
        * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 70-71
theorem enstrophy_bound (D : ℝ) (N : ℕ) (v : ℕ → ℂ) :
    2 * D * enstrophy N v ≤ ham3 D N v + (2 * energy N v) ^ (3 / 2 : ℝ) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 323-328
theorem enstrophy_bound_along_flow (D : ℝ) (N : ℕ) (u : ℝ → ℕ → ℂ) (T : ℝ)
    (hsol : ∀ t ∈ Set.Icc (0 : ℝ) T, ∀ n, n < N + 2 →
      HasDerivAt (fun s => u s n) (F kk (fun m => D * kk m ^ 3) (u t) n) t)
```

```
    (hbc : ∀ t ∈ Set.Icc (0 : ℝ) T, u t (N + 2) = 0) :
    ∀ t ∈ Set.Icc (0 : ℝ) T,
      2 * D * enstrophy N (u t) ≤ ham3 D N (u 0) + (2 * energy N (u 0)) ^ (3 / 2 : ℝ) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 338-343
theorem enstrophy_le_along_flow (D : ℝ) (hD : 0 < D) (N : ℕ) (u : ℝ → ℕ → ℂ) (T : ℝ)
    (hsol : ∀ t ∈ Set.Icc (0 : ℝ) T, ∀ n, n < N + 2 →
      HasDerivAt (fun s => u s n) (F kk (fun m => D * kk m ^ 3) (u t) n) t)
    (hbc : ∀ t ∈ Set.Icc (0 : ℝ) T, u t (N + 2) = 0) :
    ∀ t ∈ Set.Icc (0 : ℝ) T,
      enstrophy N (u t) ≤ (ham3 D N (u 0) + (2 * energy N (u 0)) ^ (3 / 2 : ℝ)) / (2 * D) := ...
```

Sharpness checks. (i) The term $(2E)^{3/2}$ cannot be dropped. Take $N = 0$, $D = 1$, $v = (1, -i, 0, \dots)$. Then $2D\Omega = |v_0|^2 + 4|v_1|^2 = 5$, while $H_3 = 1 + 4 + \mathrm{Im}(1 \cdot (-i)) = 4$ (`drop_cubic_term_fails`). (ii) The 2^{-n} factor in q_n is needed. At $v = (1, 1, 0, \dots)$, $N = 0$, $D = 1$, the rate with $q_n = \omega_n$ is $8 \neq 0$ (`rate_v11_wrong_weight`). (iii) The truncation is needed: with $v_2 = 1$ the correct-weight rate is nonzero (`rate_v111_no_truncation`).

```
-- Autoresearch_Enstrophy.lean (prior round) lines 84-86
theorem drop_cubic_term_fails :
    ¬ (2 * 1 * enstrophy 0 (fun n => if n = 0 then (1 : ℂ) else if n = 1 then -Complex.I else 0)
        ≤ ham3 1 0 (fun n => if n = 0 then (1 : ℂ) else if n = 1 then -Complex.I else 0)) := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 377-379
theorem rate_v11_wrong_weight :
    dHam (fun n => 1 * kk n ^ 3) (fun n => (1 / 2 : ℝ) ^ n * kk n) 0 v11
      (F kk (fun m => 1 * kk m ^ 3) v11) = 8 := ...
```

```
-- Autoresearch_EnstrophyFlow.lean lines 387-389
theorem rate_v111_no_truncation :
    dHam (fun n => (1 / 2 : ℝ) ^ n * (1 * kk n ^ 3)) (fun n => (1 / 2 : ℝ) ^ n * kk n) 0 v111
      (F kk (fun m => 1 * kk m ^ 3) v111) ≠ 0 := ...
```

Corollary 1 ([C]). *For $D > 0$, $\Omega(t) \le \Omega(0) + S(0)^{3/2}/D$.* Proof. *By Step 6, $H_3(0) \le 2D\Omega(0) + S(0)^{3/2}$. Insert this in* (4). □

The right side does not depend on N *as a formula*. It is a cutoff-uniform bound only for families of initial data with N-uniform $\Omega(0)$ and $S(0)$, an H^1-type class. It degenerates as $D \to 0$.

Remark 2 ($\sigma = 2$, [C]). For $\omega_n = Dk_n^2$ the same computation gives $q_n = Dk_n$, so H_2 controls $\sum k_n |v_n|^2$, not Ω. On a state with all of $S = 1$ in shell n the cubic part vanishes and $2D\Omega/(H_2 + S^{3/2}) = D4^n/(D2^n + 1)$, which tends to 2^n once $D2^n \gg 1$.

Remark 3 (Relation to the real dyadic model, [C]/[L]). For real v and $D = 0$, (1) reads $\dot v_0 = -v_0 v_1$ and $\dot v_n = 2^{n-1}(v_{n-1}^2 - 2v_n v_{n+1})$. This is one half of Waleffe's dyadic inviscid Burgers chain [eq. (12) of [2]] with $j_0 = 0$, so $v(t) = a(t/2)$. Waleffe proves energy conservation for such chains, and finite-time blow-up of every H^α norm, $\alpha > 0$, for the *infinite* chain. This needs non-negative data that also carry enough energy at high shells, $E_{B(J)}(0) \ge q^J$ for some large J and $\lambda^{-2} < q < 1$ [Theorem 1 of [2]] [L]. Our statements are at fixed finite N and say nothing about that limit. On real data $T_n \equiv 0$, so the cubic part of H sees only the phases.

3 The Lean formalization

Files and environment. The claimed module is `results/autoresearch/2026-10-25/H7/contribution/Autoresearch_EnstrophyFlow.lean` (sha256 `4cd746897f58f7a18eb27e95f703b43d49ec0eb5c5f38753cd5f0bc1a99a8dd4`). It imports the owner's compiled `ShellHamiltonian` (QuantumFluids commit `10f74daf1b39`, qf-lake build) and is compiled by the Lean v4.34.0-rc2 binary with `LEAN_PATH` set to that build (`run_lean.py`). The producer's final compile had rc 0 in 15.5 s, with linter warnings

only. The paper-stage recompile had rc 0 in 18.5 s, and its `#print axioms` output was identical [X, SE-X-0005]. The prior-round file `H7/Autoresearch_Enstrophy.lean` (sha256 `c7c9f24cfe2e623f36600d11e11841e7cbd75f4b3801fa602681980735c97f03`) holds `enstrophy_le` and `drop_cubic_term_fails`. Its recheck compile had rc 0; it took 2163.9 s because the page cache was cold, not because the proof is costly. At paper stage, `grep -c "sorry\|admit\|native_decide\|^axiom"` on the claimed file returns 0.

Owner definitions (verbatim, QuantumFluids).

The qf-lake `.trace` files record that the imported `.olean`s were built from these same `lean_src` paths. The sources were last modified (2026-09-19 23:09 UTC) before the `.olean`s were written (23:31).

```
-- QuantumFluidsShell.lean  sha256 8da44064789a2489f51993ff8b30c70ebc2a2c679e6a7824992d8f7be71a731f
-- lines 42-45
noncomputable def shellBc (k : ℕ → ℝ) : ℕ → (ℕ → ℂ) → ℂ
  | 0,     v => -((k 0 : ℂ) * (starRingEnd ℂ) (v 0) * v 1)
  | m + 1, v => (k m : ℂ) * (v m * v m)
                  - (k (m + 1) : ℂ) * (starRingEnd ℂ) (v (m + 1)) * v (m + 2)
```

New definitions (verbatim).

```
-- Autoresearch_EnstrophyFlow.lean  sha256 4cd746897f58f7a18eb27e95f703b43d49ec0eb5c5f38753cd5f0bc1a99a8dd4
-- lines 40-40
noncomputable def kk (n : ℕ) : ℝ := (2 : ℝ) ^ n
-- lines 43-44
noncomputable def energy (N : ℕ) (v : ℕ → ℂ) : ℝ :=
  (1 / 2) * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n)
-- lines 47-48
noncomputable def enstrophy (N : ℕ) (v : ℕ → ℂ) : ℝ :=
  (1 / 2) * ∑ n ∈ Finset.range (N + 2), kk n ^ 2 * Complex.normSq (v n)
-- lines 52-55
noncomputable def ham3 (D : ℝ) (N : ℕ) (v : ℕ → ℂ) : ℝ :=
  ∑ n ∈ Finset.range (N + 2), (1 / 2 : ℝ) ^ n * (D * kk n ^ 3) * Complex.normSq (v n)
    + ∑ n ∈ Finset.range (N + 1),
        ((1 / 2 : ℝ) ^ n * kk n) * ((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).im
-- lines 91-92
noncomputable def F (k ω : ℕ → ℝ) (v : ℕ → ℂ) (n : ℕ) : ℂ :=
  shellBc k n v - Complex.I * (ω n : ℂ) * v n
-- lines 95-96
noncomputable def R (v : ℕ → ℂ) (n : ℕ) : ℝ :=
  ((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).re
-- lines 99-102
noncomputable def hamG (q g : ℕ → ℝ) (N : ℕ) (v : ℕ → ℂ) : ℝ :=
  ∑ n ∈ Finset.range (N + 2), q n * Complex.normSq (v n)
    + ∑ n ∈ Finset.range (N + 1),
        g n * ((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).im
-- lines 105-109
noncomputable def dHam (q g : ℕ → ℝ) (N : ℕ) (v w : ℕ → ℂ) : ℝ :=
  ∑ n ∈ Finset.range (N + 2), q n * (2 * ((starRingEnd ℂ) (v n) * w n).re)
    + ∑ n ∈ Finset.range (N + 1), g n *
        (2 * (starRingEnd ℂ) (v n) * (starRingEnd ℂ) (w n) * v (n + 1)
          + (starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * w (n + 1)).im
```

Main statements. These are listed verbatim beside the steps of Section 2, with proof bodies replaced by ... and source line numbers given. The consistency witness is:

```
-- Autoresearch_EnstrophyFlow.lean lines 351-355
theorem zero_trajectory_is_solution (D : ℝ) (N : ℕ) (T : ℝ) :
    (∀ t ∈ Set.Icc (0 : ℝ) T, ∀ n, n < N + 2 →
      HasDerivAt (fun s => (fun (_ : ℝ) (_ : ℕ) => (0 : ℂ)) s n)
        (F kk (fun m => D * kk m ^ 3) ((fun (_ : ℝ) (_ : ℕ) => (0 : ℂ)) t) n) t)
    ∧ (∀ t ∈ Set.Icc (0 : ℝ) T, (fun (_ : ℝ) (_ : ℕ) => (0 : ℂ)) t (N + 2) = 0) := ...
```

Axiom audit (verbatim output of the final compile). All 17 audited declarations of the claimed file print the allowlist (all in allowlist: yes) [A, SE-A-0002..0006]. Line breaks inside the bracket were joined:

```
'QuantumFluids.AutoresearchEnstrophy.enstrophy_bound' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.re_conj_mul_F' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.dT_F' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.quad_tel' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.rate_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.ham_rate_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.energy_rate_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.hasDerivAt_hamG' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.const_on_Icc' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.ham_conserved' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.energy_conserved' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.enstrophy_bound_along_flow' depends on axioms: [propext, Classical.choice,
    Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.enstrophy_le_along_flow' depends on axioms: [propext, Classical.choice, Quot.
    sound]
'QuantumFluids.AutoresearchEnstrophy.zero_trajectory_is_solution' depends on axioms: [propext, Classical.choice,
    Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.rate_v11_correct' depends on axioms: [propext, Classical.choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophy.rate_v11_wrong_weight' depends on axioms: [propext, Classical.choice, Quot.
    sound]
'QuantumFluids.AutoresearchEnstrophy.rate_v111_no_truncation' depends on axioms: [propext, Classical.choice, Quot.
    sound]
```

The owner theorems used print the same footprint at paper stage (`compile_owner_axioms.json`) [A, SE-A-0001].

Planted controls. They live in a separate file (sha256 `7a3904240f4734df1360471af1bce7fb420ff27 9298298da6effc9f5c9e065ce`), which defines its own vector field `Fc` and does not import the claimed module. Both state a false energy rate:

```
-- Autoresearch_EnstrophyFlowControls.lean  sha256 7
    a3904240f4734df1360471af1bce7fb420ff279298298da6effc9f5c9e065ce  lines 13-23
theorem ctl_sorry (k ω : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) :
    ∑ n ∈ Finset.range (N + 2), 2 * ((starRingEnd ℂ) (v n) * Fc k ω v n).re = 1 := by
  sorry

/-- Planted axiom: must show `ctl_flow_cheat`. -/
axiom ctl_flow_cheat : ∀ (k ω : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ),
  ∑ n ∈ Finset.range (N + 2), 2 * ((starRingEnd ℂ) (v n) * Fc k ω v n).re = 1

theorem ctl_axiom (k ω : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) :
    ∑ n ∈ Finset.range (N + 2), 2 * ((starRingEnd ℂ) (v n) * Fc k ω v n).re = 1 :=
  ctl_flow_cheat k ω v N
```

The file compiles with rc 0. The audit catches both holes [A, SE-A-0007]:

```
'QuantumFluids.AutoresearchEnstrophyFlowControls.ctl_sorry' depends on axioms: [propext, sorryAx, Classical.
    choice, Quot.sound]
'QuantumFluids.AutoresearchEnstrophyFlowControls.ctl_axiom' depends on axioms: [propext, Classical.choice, Quot.
    sound, QuantumFluids.AutoresearchEnstrophyFlowControls.ctl_flow_cheat]
```

So the exit code is not the gate; the axiom list is. These controls show that the audit detects holes in this environment. The evidence for the claimed file is that file's own `#print axioms` output.

4 Physics: what the theorem says and does not say

System. A dyadic shell model is a caricature of the Fourier energy cascade: one complex amplitude per octave, coupled only to its neighbours [2]. Here S plays the role of energy and Ω of enstrophy. The term $-i\omega_n v_n$ is a linear dispersive phase rotation. Quantum pressure in a Gross–Pitaevskii fluid has $\omega \propto k^2$ ($\sigma = 2$). The $\sigma = 3$ law is a regulator proposed in the QuantumFluids design memo [1]. It is not derived here from a physical fluid [C].

Regime and units. The model is inviscid, unforced, finite-dimensional and dimensionless. One scaling removes the amplitude, so results are stated at $S(0) = 1$ with $D = 0.05$ [C].

What it says. At every fixed cutoff, along every solution that exists on $[0, T]$, the enstrophy stays below a bound fixed by the initial data, (4) [A]. The bound also holds for every real D (for $D \le 0$ it carries no information on Ω). By Corollary 1 it is uniform in N and t for H^1-type data at fixed $D > 0$ [C].

What it does not say. It does not prove that solutions exist; that is a hypothesis [A]. Global existence at fixed N should follow from conservation of S in finite dimensions [C], but this is not formalised. It says nothing about $N \to \infty$, blow-up or regularity of any PDE. It does not say that a physical quantum fluid has $\sigma = 3$; quantum pressure has $\sigma = 2$, which does not reach the enstrophy (Remark, Section 2). The owner's memo also records that no invariant of the form "enstrophy + positive quartic" exists in this model [1]. The bound comes from dispersion, not from a coercive interaction energy.

5 rusty-SUNDIALS numerics

All of this section is [X]. These runs illustrate and stress-test the Lean statements; they support no claim.

Method. The module `crates/qf-shell-cascade/src/sigma.rs`, with the binary `sigma3_study`, implements (2) with shells $0..n_{\max}$, so Lean $N = n_{\max} - 1$. It also implements S, Ω and H_σ with the Lean weights taken literally, the exact first variation and an analytic Jacobian (unit-tested against central differences). It uses this workspace's CVODE [5, 6]: BDF, one-step mode, atol $= 10^{-3}$ rtol. Every invariant is evaluated at every internal step, and suprema are over internal step points. The H drift is measured relative to $|Q| + \sum |\text{cubic terms}|$ at $t = 0$. Per the stage-2 logs, the crate's 32 unit tests and 2 positive-control tests pass and clippy and fmt are clean. The independent AI-agent audit (`AUDIT.md`) reran `cargo test` (32 unit and 2 positive-control tests pass) and the full study, whose summary matched `summary.json` apart from wall times. The study took 143.7 s on one core. Its 5 summary gates (computed by `summarise.py`) all pass. The cost driver is the top-shell rotation $D\,8^{n_{\max}}$. It lies on the imaginary axis, where BDF must resolve it. The largest run took 6,325,180 steps ($n_{\max} = 7$), so the sweep stops there.

Controls (Table 1). Positive control: $\sigma = 3$, decaying data $|v_n| \propto 2^{-1.5n}$, $T = 2$. From rtol 10^{-6} to 10^{-10} the drift of S falls from 5.4×10^{-7} to 4.0×10^{-10} ($n_{\max} = 4$) and from 1.2×10^{-6} to 6.9×10^{-10} ($n_{\max} = 6$). The drift of H_3 falls from 2.6×10^{-5} to 1.9×10^{-8} and from 6.3×10^{-4} to 3.7×10^{-7}. Negative control: the outgoing coupling k_1 in B_1 multiplied by 1.01. The drift does not converge: it stays at $2.6 \times 10^{-3}/4.6 \times 10^{-3}$ in S ($n_{\max} = 4/6$) and at 2.3×10^{-3} in H_3 ($n_{\max} = 4$) at every rtol, and in H_3 at $n_{\max} = 6$ it lies between 5.4×10^{-3} and 5.9×10^{-3} (Table 1). At rtol 10^{-10} that is 6.6×10^6 times the positive drift in S, and 1.2×10^5 to 1.5×10^4 times in H_3 [X, SE-X-0001].

Truncation sweep. For $\sigma = 3$, $n_{\max} = 2..7$ and 2 phase seeds, $\sup_t 2D\Omega(t)/(H_3(0) + S(0)^{3/2})$ lies between 0.249 and 0.360 with no trend in N. In 12/12 runs the supremum exceeds the $t = 0$ value, so the dynamics move the ratio. The maximum drifts are 2.1×10^{-9} in S and 4.0×10^{-6} in H_3 (Figure 2, left) [X, SE-X-0002].

$\sigma = 2$ control (Table 2). On top-shell data ($v_{n_{\max}} = 1$, other shells 10^{-3}, $T = 0.5$) the $\sigma = 2$ ratio grows from 0.667 to 1004.4. Its local $\log_2$ steps for $n_{\max} = 6 \to 10$ are 1.18, 1.10, 1.05, 1.03, in line with the Remark's $D4^n/(D2^n + 1)$. On this data the supremum equals the $t = 0$ value to within a relative

Table 1: Maximum relative drift over $t \in [0, 2]$ (internal steps), $\sigma = 3$, $D = 0.05$. Generated from `data/summary.json` by `gen_numbers.py`.

$n_{\max}$	Lean N	rtol	pos. ΔS	pos. ΔH_3	neg. ΔS	neg. ΔH_3	BDF steps
4	3	10^{-6}	5.4×10^{-7}	2.6×10^{-5}	2.6×10^{-3}	2.3×10^{-3}	5,405
4	3	10^{-8}	1.8×10^{-8}	8.6×10^{-7}	2.6×10^{-3}	2.3×10^{-3}	7,302
4	3	10^{-10}	4.0×10^{-10}	1.9×10^{-8}	2.6×10^{-3}	2.3×10^{-3}	15,460
6	5	10^{-6}	1.2×10^{-6}	6.3×10^{-4}	4.6×10^{-3}	5.9×10^{-3}	188,969
6	5	10^{-8}	3.2×10^{-8}	1.7×10^{-5}	4.6×10^{-3}	5.5×10^{-3}	408,026
6	5	10^{-10}	6.9×10^{-10}	3.7×10^{-7}	4.6×10^{-3}	5.4×10^{-3}	878,138

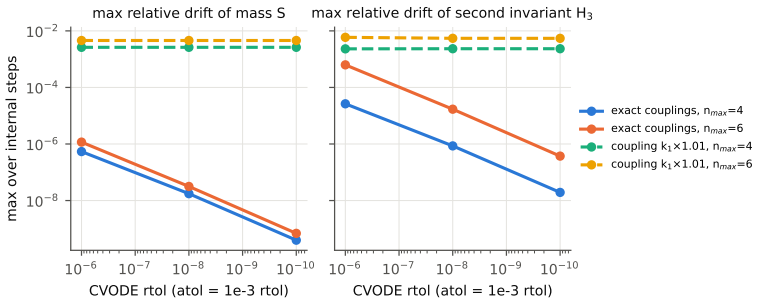


Figure 1: Drift against rtol for the positive control (converges) and the perturbed-coupling negative control (does not) [X]. Generated by `make_figures.py` from `sigma3_study.json`.

5.2×10^{-7} for $\sigma = 2$ (absolute 3.2×10^{-7} for $\sigma = 3$), so the growth is static, not dynamical. On the same data $\sigma = 3$ reaches at most 0.995 and never exceeds 1. On cascading (decaying) data, H_2 is conserved to 8.2×10^{-10} ($n_{\max} = 4$) and 9.9×10^{-11} ($n_{\max} = 6$), consistent with conservation for every ω. Meanwhile the $\sigma = 2$ enstrophy grows to 19.4 and 32.6, and the analogue ratio reaches 1.57 and 2.43 [X, SE-X-0003].

Table 2: Top-shell data, sup ratio $2D\Omega/(H_\sigma(0) + S(0)^{3/2})$, $D = 0.05$, rtol 10^{-10}. "supp." rows are supplementary probes with the same binary and seeds (`sigma2_probes.jsonl`).

$n_{\max}$	Lean N	$\sigma = 2$	$\sigma = 3$	source
2	1	0.6667	0.444	C2
3	2	2.286	0.762	C2
4	3	7.111	0.928	C2
5	4	19.69	0.981	C2
6	5	48.76	0.995	C2
7	6	110.7	not run	supp.
8	7	237.4	not run	supp.
9	8	492.8	not run	supp.
10	9	1004	not run	supp.

6 HuggingFace dataset illustration

Dataset. We use `polymathic-ai/turbulent_radiative_layer_2D`, revision `2ee7756575ff6f9098 1d0308cbcb6a2ab5995bdc`, licence CC-BY-4.0 per its card. It is part of The Well [3] and was generated with Athena++. It solves 2D compressible Euler with radiative cooling, periodic in x and zero-gradient in y, and its associated paper is [4] [L, SE-L-0002]. We downloaded one test file,

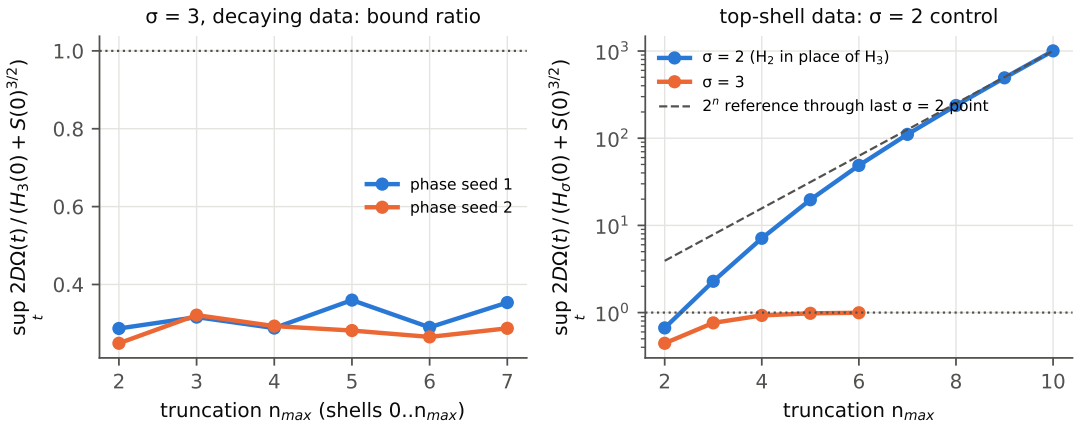


Figure 2: Left: $\sigma = 3$ bound ratio against truncation for decaying data. Right: top-shell data, $\sigma = 2$ (grows like 2^n) against $\sigma = 3$ (≤ 1). Dotted: the Lean bound, ratio $= 1$ [X].

`data/test/turbulent_radiative_layer_tcool_0.32.hdf5` (109 MB, sha256 `06cbd36c1dbea84e70906a363d20143b228765ee37850ec65351e247cf7df747`), and the card and YAML files.

What we computed. For 4 developed snapshots (time indices 25, 50, 75, 100) we take the FFT of the velocity along the periodic x axis only, remove $k_x = 0$, average over y, and sum $k_x \in [2^n, 2^{n+1})$ into shells $n = 0..5$. Pipeline controls pass. Parseval holds to 4.0×10^{-16}, a synthetic $\sin(8x)$ field lands entirely in shell 3, and white noise gives $\log_2$ slope 1.01. The data slopes are -2.13, -1.56, -1.79, -1.81. The moduli, normalised to $S = 1$ with 2 random phase seeds, initialise 24 $\sigma = 3$ runs ($n_{\max} = 5, 6, 7$, upper shells zero). The bound holds in all of them, with maximum ratio 0.631. The supremum exceeds the $t = 0$ value in 24/24. Drifts are $\leq 2.4 \times 10^{-8}$ in S and $\leq 1.2 \times 10^{-6}$ in H_3 (Figure 3) [X, SE-X-0004].

Caveats. The data provide a realistic spectral shape, nothing more. The flow is compressible and cooling, and does not conserve energy. The spectrum is 1D, uses velocity only (not $\sqrt{\rho}\,u$) and carries no phases. The phases are random. Over $T = 2$ the zero-initialised top shells barely fill, so the ratio is the same at $n_{\max} = 5, 6, 7$ and these runs do not probe cutoff dependence.

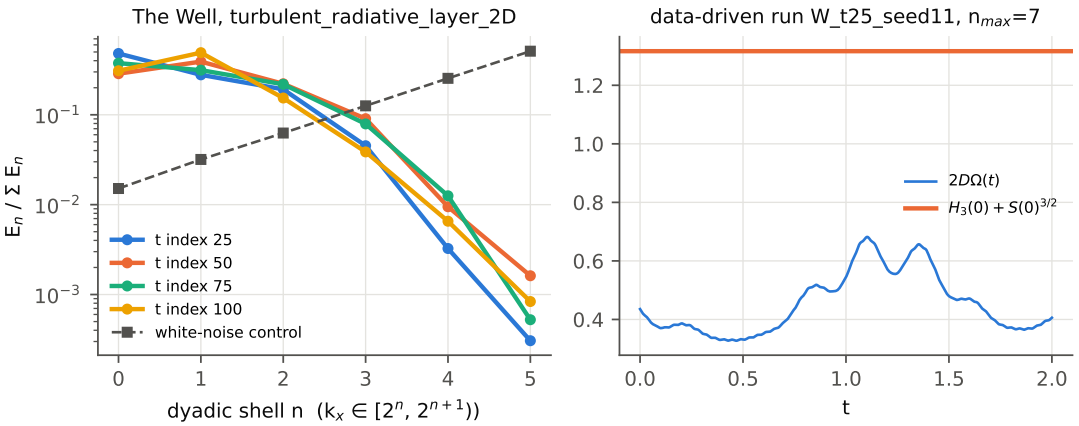


Figure 3: Left: dyadic shell spectra of The Well snapshots and a white-noise control. Right: one data-initialised run, $2D\Omega(t)$ against the Lean bound [X].

7 Limitations

- *Verification.* The producer checked the files, another AI agent instance recompiled them at paper stage, and a separate AI-agent instance ran an independent audit (`AUDIT.md`). It recompiled the claimed and prior files (rc 0, every audited declaration within the allowlist), planted its own `sorry` and axiom and saw the axiom audit flag exactly the dependent theorems, checked statement fidelity against `#print` of the owner definitions, and reran the numerics and tests. No human has audited statement adequacy (all Tier A ledger rows carry `audit: null`).
- *Solution hypothesis.* Existence is assumed. The hypothesis uses two-sided `HasDerivAt` at every $t \in [0, T]$, endpoints included. For $T < 0$ the interval is empty and the along-flow statements are vacuous. The zero trajectory satisfies it (`zero_trajectory_is_solution`); this shows consistency, not non-triviality.
- *Fixed N.* There is no infinite-shell statement. Cutoff uniformity needs N-uniform $\Omega(0)$ and $S(0)$ (Corollary 1, Tier C).
- *Definitions.* The model is the owner's conjugated complexification plus dispersion. Its physical fidelity is the owner's design choice and is not certified here.
- *Numerics.* $n_{\max} \leq 7$, a single D, finite T, suprema at step points. The BDF damping of fast rotations gives a negative S drift (largest -2.8×10^{-5} for $\sigma = 3$ top-shell data at $n_{\max} = 6$, -1.3×10^{-4} for $\sigma = 2$ at $n_{\max} = 10$). Damping lowers Ω and so biases runs toward passing, but it cannot explain the passes. In the sweep and the data runs the margin to the bound is at least $1 - 0.631$, far above the drifts. For the top-shell $\sigma = 3$ runs (up to 0.995) the supremum exceeds the $t = 0$ value by at most 3.2×10^{-7}, far below the margin $1 - 0.995$ to the bound. Larger N would need an exponential or splitting integrator, which was not attempted.
- *Scope.* The open second half of the memo's step (the "O5 trap" argument) was not attempted. No novelty is claimed for the inequality, which is a corollary of the owner's lemma. A literature check on complex dyadic and SHG-chain models, listed as pending in the memo, was not completed here.

8 Reproducibility

Lean (from the AutoevolveAI worktree; `OUT.json` is any output path):

```
taskset -c 4-7 .venv/bin/python -I results/autoresearch/2026-10-25/H7/contribution/run_lean.py \
  results/autoresearch/2026-10-25/H7/contribution/Autoresearch_EnstrophyFlow.lean OUT.json
python -I results/autoresearch/2026-10-25/H7/contribution/paper_stage_recompile.py
```

Rust: rusty-SUNDIALS worktree, branch `physics/ar-sigma3-shell-2026-10-25`, based on origin/main `47ef5044c10b04c64d8c86f483f1aa47c400ab28`. Code commit `99f5df89e8d8380ea1a9f065ae87308e779224fc`, results `e869e5fc4e38df24bc93ee2c22e19f2e68920041`, $\sigma = 2$ probes `dfa9a0880450a7601e1ecdcb735b5897c6319ffd`. These commits are not pushed at the time of writing.

```
CARGO_TARGET_DIR=/home/callensxavier_gmail_com/rusty-target-ar cargo test --release -j 2 --offline \
  --manifest-path WT/Cargo.toml -p qf-shell-cascade -- --test-threads=2
venv-pta/bin/python -I R/scripts/well_spectrum.py \
  hf-data/sigma3_enstrophy/data/test/turbulent_radiative_layer_tcool_0.32.hdf5 R/well_spectrum.json   # needs
    h5py
python -I R/scripts/make_amps.py R/well_spectrum.json R/well_amps.txt
taskset -c 4-7 /home/callensxavier_gmail_com/rusty-target-ar/release/sigma3_study full R/sigma3_study.json \
  R/well_amps.txt 99f5df89e8d8380ea1a9f065ae87308e779224fc
python -I R/scripts/run_probes.py /home/callensxavier_gmail_com/rusty-target-ar/release/sigma3_study R/
    sigma2_probes.jsonl
python -I R/scripts/summarise.py R/sigma3_study.json R/summary.json
python -I R/scripts/make_figures.py R/sigma3_study.json R/well_spectrum.json R/figures R/sigma2_probes.jsonl
```

Here `WT` is the rusty-SUNDIALS worktree and `R` is `WT/crates/qf-shell-cascade/results/sigma3`. Absolute paths are in `H7/contribution/NUMERICS.md`. Paper: `python -I gen_numbers.py; python -I make_listings.py; pdflatex main.tex` (twice); ledger: `python -I ledger/build_ledger.py`.

9 Claim ledger

`ledger/ledger.json` holds 7 Tier A, 2 Tier L, 4 Tier C and 5 Tier X rows. Each row points to a content-addressed evidence blob in `ledger/evidence/`. Tier A blobs hold verbatim axiom lines with source and `.olean` hashes. Tier L blobs hold the excerpts, Tier C blobs the argument, and Tier X blobs the result-file hashes with `claim_licence: false`. The Elenchus gate (`tools/ledger.py ledger.json -evidence-dir evidence`) was run on it. Its output is in `ledger/gate_output.txt`: exit code 1, no blocking findings, and one `LEDGER_UNAUDITED_TIER_A` flag per Tier A row, because no person has audited statement adequacy. The Waleffe Theorem 1 source text used by the auditor's Elenchus `citation_check.py` run is persisted in `ledger/sources/` with its claim file; the rerun there returns ACCEPTED (exit 0, tier ceiling L), recorded in `ledger/sources/citation_check_output.txt`.

References

[1] SocrateAI-Scientific-QuantumFluids, project repository (owner's Lean sources `lean_src/QuantumFluidsShell.lean`, `lean_src/ShellHamiltonian.lean`, design memo `docs/DUAL_SCALE_PROPOSAL.md` §5, §7), commit `10f74daf1b39d2ac51a785452329fff0003cfc83`. Project document, not peer reviewed.

[2] F. Waleffe, *On some dyadic models of the Euler equations*, arXiv:math/0410380 (2004).

[3] R. Ohana, M. McCabe, et al., *The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning*, arXiv:2412.00568 (2024).

[4] D. B. Fielding, E. C. Ostriker, G. L. Bryan, A. S. Jermyn, *Multiphase gas and the fractal nature of radiative turbulent mixing layers*, Astrophys. J. Lett. 894, L24 (2020), doi:10.3847/2041-8213/ab8d2c (as given in the dataset card; not read by us).

[5] D. J. Gardner, D. R. Reynolds, C. S. Woodward, C. J. Balos, *Enabling New Flexibility in the SUNDIALS Suite of Nonlinear and Differential/Algebraic Equation Solvers*, ACM Trans. Math. Softw., arXiv:2011.10073 (2020) (authors, title and identifier verified; cited for the SUNDIALS/CVODE design only).

[6] rusty-SUNDIALS, Rust workspace re-implementing SUNDIALS integrators; branch and commits as in Section 8.

[7] X. Callens, *Planar universal optimality, tiered: kernel-checked consequences, a non-uniqueness theorem, and a CVODE N-body consistency check for the Yukawa interaction*, v2, doi:10.5281/zenodo.23293429; cited for the tiering and ledger method.

[8] X. Callens, student textbook on triangular-lattice universal optimality (2026), doi:10.5281/zenodo.23295142; cited for the tiering method.